codehive

Mastering LIFO Data Architecture

1. What is a Stack?

A **Stack** is a linear data structure that follows the **Last-In-First-Out (LIFO)** principle. It mirrors real-world scenarios like a stack of pancakes or a pile of cafeteria trays.

> In LIFO, the most recently added element is the first one to be removed. You can only interact with the "Top" of the stack.

2. Core Stack Operations

There are five fundamental actions associated with a professional Stack implementation:

- **Push:** Adds a new element to the top.
- **Pop:** Removes and returns the top element.
- **Peek:** Views the top element without removing it.
- **isEmpty:** Checks if the stack has no elements.
- **Size:** Returns the total count of items.

3. Implementation: Python Lists

Python's built-in `list` is an excellent starting point for stacks because `append()` and `pop()` are optimized for end-of-sequence operations.

```
stack = []
stack.append('A')  # Push
top = stack[-1]    # Peek
val = stack.pop()  # Pop
```

This method is memory-efficient and simple to implement.

4. The Object-Oriented Approach

For large-scale applications, encapsulating a list inside a `Stack` class provides better control and cleaner code.

```
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop() if not self.isEmpty() else None
    def isEmpty(self):
        return len(self.items) == 0
```

5. Implementation: Linked Lists

Unlike arrays, a Linked List based stack uses "Nodes". Each node stores a value and a pointer to the next element.

Advantage: Dynamic size. It only uses as much memory as it needs at any given moment. Nodes can be scattered anywhere in the physical RAM.

6. Linked List Node Structure

In a Linked List stack, the "Head" always represents the "Top".

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

class LLStack:
    def __init__(self):
        self.head = None
```

7. Array vs. Linked List

Feature	Array/List	Linked List
Memory	Contiguous	Non-contiguous
Access	Fast (Index)	Slow (Traversal)
Sizing	Fixed/Resizing	Fully Dynamic

8. Real-World Applications

Stacks are not just theoretical; they empower everyday software:

- **Undo/Redo:** Text editors store your changes on a stack.
- **Browser History:** Navigating "Back" pops the last URL.
- **Call Stack:** Managing function execution in programming languages.

9. Backtracking Algorithms

Algorithms like **Depth-First Search (DFS)** use stacks to remember which paths to return to after hitting a dead end in a maze or graph.

```
Algorithm visualized
Path: A → B → C → [Dead End]
Action: Pop C
Current: B → [Exploring New Branch]
```

10. Mastery Summary

- ✓ Mastered LIFO (Last-In, First-Out).
- ✓ Understood Push, Pop, and Peek.
- ✓ Compared List-based vs. Node-based storage.
- ✓ Recognized Stacks in Undo/Redo logic.
- ✓ Ready for higher-order DFS algorithms.

codehive

Data Architect Series | Module 03